📘 Greenfield Documentation Template

License Management System (LMS)

1. Problem Statement

Objective

Design a B2B License Management System where enterprise customers purchase software licenses in bulk. Administrators can distribute and revoke licenses for employees while ensuring the total number of active licenses never exceeds the purchased capacity.

The SaaS application should also be able to verify whether an employee currently owns an active license.

---

2. Functional Requirements

FR-1 Create License Pool

- Create a license pool
- Specify total capacity
- Specify start date
- Specify end date

FR-2 Assign License

Administrator should

- Search employee
- Check available seats
- Allocate seat
- Store assignment
- Record audit log

FR-3 Revoke License

Administrator can

- Select employee
- Remove active license
- Return seat to pool
- Record audit log

FR-4 Verify License

SaaS application checks

Employee Email

↓

Is license active?

↓

YES / NO

---

3. Non Functional Requirements

- Scalability
- Concurrency
- Security
- High Availability
- Auditability
- Performance
- Data Consistency
- ACID Transactions

---

4. Actors

Enterprise Admin

Employee

SaaS Application

Database

---

5. System Constraints

Pool Capacity

Assigned Seats ≤ Purchased Capacity

Active Date

Today must be

Start Date ≤ Today ≤ End Date

Double Allocation

Two admins assigning the last seat

↓

Only one succeeds

---

6. Assumptions

- Employee email is unique
- Admin belongs to one organization
- One employee has one active seat per pool
- Database supports transactions
- Audit history is immutable

---

7. Domain Model

Company

↓

LicensePool

↓

SeatAssignment

↓

Employee

↓

AuditLog

↓

Administrator

---

8. High Level Architecture

Browser

↓

React Admin UI

↓

REST API

↓

Business Layer

↓

Repository Layer

↓

PostgreSQL

Components

Admin UI

Creates pools

Assigns seats

Revokes seats

API

Receives requests

Validates

Calls services

Service

Business Rules

Transactions

Validation

Concurrency

Repository

Database operations

Database

Persistent storage

---

9. Folder Structure

src/

controller/

service/

repository/

entity/

dto/

config/

exception/

security/

util/

tests/

---

10. Database Design

Administrator

admin_id

name

email

Employee

employee_id

email

name

LicensePool

pool_id

company_id

capacity

assigned_count

start_date

end_date

status

SeatAssignment

assignment_id

pool_id

employee_id

assigned_by

assigned_at

status

AuditLog

audit_id

action

performed_by

employee_id

timestamp

---

11. Relationships

Company

1

↓

Many

LicensePool

↓

Many

SeatAssignment

↓

One

Employee

↓

Many

AuditLog

---

12. API Design

Create Pool

POST

/license-pools

Request

capacity

startDate

endDate

Response

poolId

capacity

availableSeats

Assign Seat

POST

/license-pools/{id}/assign

Request

employeeEmail

Response

assignmentId

employee

status

Errors

409 Pool Full

400 Invalid

403 Expired Pool

Revoke

POST

/license-pools/{id}/revoke

Verify

GET

/licenses/verify

?email=abc@company.com

Response

licensed

expiry

pool

---

13. Business Rules

Assign

Pool Active?

↓

Available Seat?

↓

Already Assigned?

↓

Allocate

Revoke

Assignment Exists?

↓

Is Active?

↓

Deactivate

↓

Increase Seat Count

Verify

Employee Exists?

↓

Assignment Active?

↓

Pool Active?

↓

Licensed

---

14. Request Lifecycle

Browser

↓

Controller

↓

Validation

↓

Service

↓

Repository

↓

Database

↓

Response

↓

Browser

---

15. Concurrency

Admin A

Assign

↓

Lock Row

↓

Update

↓

Commit

SUCCESS

Admin B

Assign

↓

Wait

↓

Pool Full

↓

Rollback

Solution

Database Transaction

+

SELECT FOR UPDATE

OR

Optimistic Locking

---

16. Error Handling

Pool Full

↓

409 Conflict

Pool Expired

↓

403 Forbidden

Employee Missing

↓

404

Validation Error

↓

400

---

17. Security

JWT

HTTPS

Role Based Access

Input Validation

Parameterized SQL

Audit Logs

---

18. Scalability

Future

Redis Cache

Message Queue

Read Replica

Microservices

Kubernetes

---

19. Testing

Unit Tests

Integration Tests

API Tests

Concurrency Tests

Boundary Tests

---

20. Future Enhancements

Multi Tenant

Billing

Payments

Email Notifications

Analytics Dashboard

License Expiry Reminder

Redis Cache

Event Driven Architecture